TECHNISCHE PROJEKTÜBERSICHT

Smart Energy Dashboard

Eine detaillierte Erklärung der Programmteile, ihrer Aufgaben und ihres Zusammenspiels – vom Messgerät bis zur Anzeige.

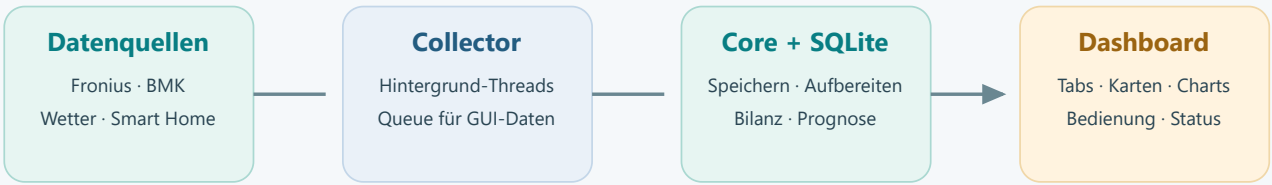
Was macht das Programm?
Es sammelt Energie-, Heizungs-, Wetter- und Smart-Home-Daten, speichert sie lokal, berechnet daraus Bilanzen und Prognosen und stellt die Ergebnisse in einer interaktiven Desktop-Oberfläche dar.

Projekt: **Dashboard**
Technologie: Python 3.11+, Tkinter / CustomTkinter, SQLite
Dokumentation anhand der aktuellen Projektstruktur

01 · ORIENTIERUNG

Das Programm als Ganzes

Die Architektur trennt Datenerfassung, fachliche Berechnung, Speicherung und Darstellung. Dadurch können Auswertungen unabhängig von den Widgets getestet und Datenquellen in der Oberfläche gemeinsam genutzt werden.



Messwerte fließen zur UI; Abfragen und Aktionen laufen über passende Clients und Module zurück.

src/main.py · Start und Betrieb

Erzeugt Fenster und Datenbank, startet Collector-Threads, vermittelt Daten an die GUI und verwaltet Schließen sowie Neustart nach einem Absturz.

src/core/ · Fachlogik

Enthält Datenquellen, Datenbankzugriff, gemeinsame Datenmodelle, Energie- und Wärmeauswertungen, Prognosen und Diagnosefunktionen.

src/ui/ · Oberfläche

Stellt App-Rahmen, Zustand, Farben, wiederverwendbare Komponenten und Visualisierungen bereit.

src/tabs/ · Produktbereiche

Jeder Tab verbindet fachliche Daten mit einer konkreten Ansicht – zum Beispiel Ertrag, Wärme, Thermostate oder Kalender.

Grundprinzip: Lange Netzwerkzugriffe laufen nicht direkt in der Zeichenroutine der Oberfläche. Die Oberfläche bleibt dadurch bedienbar, während Daten aktualisiert werden.

Projektdateien: `src/main.py`, `src/core/`, `src/ui/`, `src/tabs/`

Vom Programmstart zum Messwert

1. **Initialisierung:** `src/main.py` richtet Protokollierung und Crash-Logging ein und erstellt das Tk-Hauptfenster.
2. **Datenbank:** `DataStore` öffnet SQLite, initialisiert das Schema, importiert – falls vorhanden – CSV-Daten und kann alte Messungen verdichten und ein Backup anlegen.
3. **App-Aufbau:** `MainApp` in `src/ui/app.py` erzeugt den Energie-Bereich sowie die verfügbaren Tabs.
4. **Abfrage:** Je ein Daemon-Thread liest ungefähr alle zehn Sekunden Wechselrichter- beziehungsweise BMK-Daten aus.
5. **Speichern und Weitergeben:** Die Quellenmodule speichern Messungen im DataStore. Neue Werte werden zusätzlich in eine threadsichere Queue gelegt.
6. **GUI-Aktualisierung:** Die Hauptschleife prüft die Queue regelmäßig und ruft die Datenhandler der App im GUI-Thread auf.
7. **Abschluss:** Beim Schließen wird das Beenden signalisiert, den Collectors kurz Gelegenheit zum Stoppen gegeben, die gemeinsame Datenbank geschlossen und das Fenster zerstört.

Messquellen und gemeinsame Messwertnamen

Modul	Aufgabe und typische Werte
`core/Wechselrichter.py`	Fragt Fronius ab: PV-Leistung, Netzleistung, Batterieleistung und Ladezustand sowie weitere verfügbare Anlagenwerte.
`core/BMKDATEN.py`	Liest BMK-Heizungswerte: Kessel, Warmwasser, Außentemperatur und Pufferspeicher oben, Mitte und unten.
`core/weather.py`	Holt Wetter- und stündliche Temperaturdaten von Open-Meteo. Ein API-Schlüssel ist dafür laut Projekt nicht nötig.
`core/homeassistant.py`	Gemeinsamer Home-Assistant-Client für Zustände, Historie, Dienste, Lichter, Szenen und Webhooks.
`core/schema.py`	Definiert zentrale Schlüssel und Einheiten, beispielsweise `pv_power_kw`, `battery_soc_pct` und `buf_top_c`.

SQLite als gemeinsames Gedächtnis

`core/datastore.py` bündelt Initialisierung, Schema-Migration, Messwert-Insert, Zeitreihenabfragen, Stunden-/Tageswerte, Verdichtung und Datenbank-Backup. Die Hauptdatenbank liegt standardmäßig bei `src/core/data.db`; `DASHBOARD_DB_PATH` kann den Pfad überschreiben.

Einheitliche Zeitbasis

`core/time_utils.py` behandelt Datenbank-Zeitstempel kanonisch als UTC. Erst für Anzeige und Diagramme werden sie in die lokale Zeit umgerechnet. Das vermeidet Sortier- und Sommerzeitfehler.

Was die Energiebereiche berechnen

Elektrische Energie und PV

- **Energie-Tab:** Zentrale Live-Übersicht mit Kennzahlen, Energiefluss und Pufferspeicher-Visualisierung. Der UI-Rahmen nutzt unter anderem `EnergyFlowView`, `BufferStorageView` und eine PV-Sparkline.
- **Ertrag-Tab –** `tabs/ertrag.py`**:** Betrachtet PV-Erzeugung nach Tag und Zeitraum. Ergänzt die Messhistorie um Tagesbilanzen, Prognosewerte und deren Status.
- **Tagesproduktion –** `tabs/tagesproduktion.py`**:** Visualisiert Tages- und längerfristige PV-Produktion, inklusive auswählbarer Zeiträume und Diagramm-Updates.
- **Tagesbilanz –** `core/energy_day.py`**:** Lädt Messproben, fasst sie zusammen und berechnet Kennzahlen wie Autarkie, Eigenverbrauch und Akkuverlauf. Datenlücken werden dabei berücksichtigt.
- **Ertragsprüfung –** `core/ertrag_validator.py`**:** Prüft die Ertragshistorie und kann sie anhand gespeicherter Fronius-Daten rekonstruieren.

Wärme, Speicher und Heizung

- **Heizung –** `tabs/historical.py`**:** Zeigt historische Temperaturreihen und ermöglicht die Auswahl verschiedener Zeitfenster.
- **Wärme –** `tabs/waerme.py`**:** Zeigt Pufferspeicher, Tagesverlauf und saisonale Werte. Im Fokus stehen gespeicherte Wärme, Holzfeuerung und Solarthermie.
- **Erkennen von Heizereignissen –** `core/heating_events.py`**:** Leitet Einheizvorgänge heuristisch aus Heizungs-/Temperaturdaten ab.
- **Statistik –** `core/heating_stats.py`**:** Gruppiert Messwerte in Zeitabschnitte, klassifiziert Ereignisse und berechnet nutzbare Speicherwärme sowie Tages- und Saisonstatistiken.
- **Wärmebedarf –** `core/heat_demand.py`**:** Lernt den Zusammenhang zwischen Wärmebedarf, Außentemperatur, Solarthermie und Anlagenwerten aus den vorhandenen Daten.
- **Empfehlung –** `core/heating_forecast.py`**:** Verknüpft nutzbare Speicherenergie und geschätzten Verbrauch mit dem erwarteten Solarertrag, um einen geeigneten Einheizzeitpunkt abzuleiten.

> **Wichtig:** Prognose und Empfehlung sind modellbasierte Schätzungen, keine Messwerte. Ihre Aussagekraft hängt von Datenhistorie, Sensorqualität, Anlagenparametern und Wettervorhersage ab.

Wie die Bilanz entsteht

Messwerte werden zeitlich geordnet, auf geeignete Intervalle verdichtet und anschließend zu Tages- oder Saisonkennzahlen zusammengeführt. So lassen sich Live-Ansicht und Langzeitvergleich auf derselben Datenbasis aufbauen.

Wie Wärme eingeschätzt wird

Temperaturen im Speicher dienen als Hinweis auf den Wärmevorrat. Historischer Wärmeverbrauch und Wetter helfen einzuschätzen, wie schnell dieser Vorrat sinkt und ob Solar voraussichtlich nachliefert.

Oberflächen: `tabs/ertrag.py`, `tabs/tagesproduktion.py`, `tabs/historical.py`, `tabs/waerme.py`

04 · SMART HOME

Thermostate, Licht und Dienste

Bereich	Ansicht	Fachliche Aufgaben
Thermo	`tabs/tado.py`	Raumkarten, Thermostat-Ringe, Soll-/Istwerte, Temperaturverläufe und Bedienung. Die Quelldaten können abhängig von der Konfiguration über Home Assistant oder die direkte Tado-Anbindung kommen.
Raumklima	`core/climate.py`	Normalisiert Raumdaten aus unterschiedlichen Quellen in ein gemeinsames Modell und fasst Raumzustände zusammen.
Temperaturhistorie	`core/climate_history.py`	Speichert und liefert Temperatur-Zeitreihen; daraus können Verlaufskurven und Tagesberichte entstehen.
Heizplan	`core/climate_plan.py`	Berechnet Zeitabschnitte und Statusinformationen für Temperaturpläne und die nächste Planänderung.
Licht	`tabs/hue.py`, `core/lights.py`	Steuert Licht und Szenen über Home Assistant. Die getrennte Lichtlogik kann unabhängig von Widgets getestet werden.
Aktionen	`tabs/homeassistant_actions.py`	Stellt konfigurierbare Home-Assistant-Skripte, Automationen und andere Aktionen als gruppierte Bedienelemente dar.
Kalender	`tabs/calendar.py`	Lädt iCalendar-Feeds, zeigt Termine an und unterstützt Tages-/Monatsansichten.
Musik	`tabs/spotify.py`	Steuert Spotify-Wiedergabe, Geräte, Playlists und Lautstärke; OAuth-Hilfe liegt in `src/spotifylogin.py`.

Wie die Home-Assistant-Anbindung aufgebaut ist

`core/homeassistant.py` lädt die Konfiguration, kapselt HTTP-Aufrufe und stellt einheitliche Methoden für Zustandsabfragen, Historie, Dienstaufrufe und Webhooks bereit. Dadurch müssen Tabs nicht jedes Mal eigene Verbindungslogik implementieren. Die App-Hilfen in `ui/app_callbacks.py` bündeln zusätzlich Aktionen wie Skript-, Automations- und Anwesenheitsaufrufe.

Darstellung ist nicht Datenquelle

Der Thermostat-Tab rendert Bedienelemente und Verläufe. Das gemeinsame Klima-Modul vereinheitlicht dagegen Daten und Status, bevor UI-Code sie anzeigt.

Konfiguration statt fest eingebauter Daten

Feeds, Endpunkte und Home-Assistant-Aktionen werden aus JSON-Dateien eingelesen. Beispielkonfigurationen zeigen die erwarteten Felder mit Platzhaltern.

> Hilfe und Home-Assistant-Aktionen sind in der aktuellen App-Struktur dem Health-Bereich untergeordnet und erscheinen nicht als separate Haupttabs.

Konfigurationsvorlagen: `config/homeassistant.example.json`, `config/calendar.example.json`, `config/spotify.example.json`

05 · LERNENDE AUSWERTUNGEN

Prognosen und Warnungen

Die Prognosemodule ergänzen aktuelle Messungen um Wetterdaten und die eigene Anlagenhistorie. Sie sind getrennte Bausteine: Vorhersagen erstellen, historische Fehler messen und Modelle anhand neuer Daten nachjustieren.

Modul	Rolle in der Verarbeitung
`core/pv_forecast.py`	Kalibriert ein PV-Modell aus gespeicherten Messdaten und Wetter-/Strahlungsmerkmalen. Liefert eine zeitaufgelöste Erzeugungsvorhersage und Tages-kWh-Schätzungen.
`core/forecast_log.py`	Speichert Prognosen und vergleicht sie später mit dem tatsächlichen Verlauf. Berechnet Gütemaße und kann systematische Abweichungen bzw. Bias ermitteln.
`core/forecast_learning.py`	Archiviert Wetter- und Anlagenhistorie und stellt stündliche Reihen sowie Trainingsdaten für Lernmodelle bereit.
`core/heat_demand.py`	Enthält neben dem Wärmebedarfsmodell auch Lern-/Modellfunktionen für Solarthermie.
`core/alerts.py`	Prüft unter anderem auf ungewöhnlich geringe PV-Leistung und auffälligen Wärmebedarf; Warnungen können lokal erfasst und über einen konfigurierten Sender ausgegeben werden.

Ein typischer Prognosezyklus

1 · Eingaben
Historische Messungen, aktuelle Anlagenwerte, Wettervorhersage und geografische Anlagenparameter werden zusammengeführt.

2 · Modell
Kalibrierte Funktionen schätzen erwartete PV-Erzeugung, Solarthermie und Wärmebedarf zeitlich aufgelöst.

3 · Entscheidungshilfe
Die Wärme-Empfehlung vergleicht Restwärme und Verbrauchsrate mit dem erwarteten Solarzuwachs.

4 · Rückkopplung
Spätere Istwerte werden mit gespeicherten Prognosen verglichen und verbessern die Beurteilung der Modellgüte.

Sonnengeometrie

`core/solar_geometry.py` berechnet den Sonnenstand und die Einstrahlung auf geneigte Flächen. Diese Geometrie kann die PV-Modellierung ergänzen, indem die Orientierung der Anlage und die Sonnenposition berücksichtigt werden.

Verlässlichkeit und Grenzen

- Ein Modell ist nur so belastbar wie seine Messhistorie und die Zuordnung der Messwerte.
- Wetterprognosen und Sensorlücken beeinflussen die berechneten Ergebnisse.
- Prognosen sollten in der Oberfläche als Schätzungen erkennbar bleiben und Live-Messwerte nicht ersetzen.

Wetterclient: `core/weather.py` · saisonale Auswertung: `core/heating_stats.py` · Warnungen: `core/alerts.py`

06 · AUFBAU DER OBERFLÄCHE

App-Rahmen, Tabs und Bausteine

Der App-Rahmen

`ui/app.py` erzeugt Fensterinhalt, Energie-Startbereich und Tab-Leiste. Es verbindet UI-Zustand, Datenbank, Home-Assistant-Client, Wetterabfrage und die einzelnen Tab-Klassen. Eine `TabviewWrapper`-Kompatibilitätsschicht unterstützt ältere Notebook-Aufrufe.

Aktuelle Haupttabs

Anzeige	Modul	Einordnung
Energie	`ui/app.py` + `ui/views/`	Startübersicht und Live-Energiefluss
Licht	`tabs/hue.py`	Home-Assistant-Lichtsteuerung
Spotify	`tabs/spotify.py`	Musikwiedergabe
Thermo	`tabs/tado.py`	Thermostate und Raumklima
Kalender	`tabs/calendar.py`	iCalendar-Termine
Heizung	`tabs/historical.py`	Temperaturhistorie
Wärme	`tabs/waerme.py`	Speicher, Einheizen, Wärmeprognose
Ertrag	`tabs/ertrag.py`	PV-Ertrag und Prognose
Tagesprod.	`tabs/tagesproduktion.py`	Produktion im Zeitverlauf
Status (optional)	`tabs/status.py`	Systemstatus; konfigurierbar per Umgebungsvariable
Health	`tabs/healthcheck.py`	Datenquellen, Prüfungen, Hilfe-/Verwaltungsansichten

> **Sonderfall Analyse:** `tabs/analyse.py` wird in `ui/app.py` importiert. Im Aufbau der aktuellen Haupttabs wird sie jedoch nicht hinzugefügt; aus dem Import allein folgt daher nicht, dass die Ansicht erreichbar ist.

Wiederverwendbare UI-Schichten

`ui/components/`

Karten, Header, Statusleiste, Kennzahl-Kacheln, Icons, Gesten, Touch-Slider, Thermostat-Ring, Temperaturdiagramm und Bildschirmtastatur.

`ui/views/`

Größere Visualisierungen: Energiefluss, Energiechart, Pufferspeicher und PV-Sparkline; inklusive gemeinsamer Chart-Größenlogik.

`ui/app_state.py` + `state_schema.py`

Verwalten App-Zustand und prüfen/normalisieren Daten, die in die Oberfläche gelangen.

`ui/app_helpers.py` + Handlers

Teilen Zeitformatierung, Statusdarstellung, Messwertaufbereitung und Home-Assistant-Aufrufe in kleinere Module auf.

Die Tab-Leiste zeigt nur Module an, die importiert und im App-Aufbau tatsächlich registriert werden.

Konfiguration, Diagnose und Tests

Konfiguration und erzeugte Dateien

Ort	Inhalt / Zweck
`config/`	JSON-Konfigurationen für Home Assistant, BMK, Wetter, Spotify und Kalender. Zu vielen gibt es `*.example.json`-Vorlagen. Echte Zugangsdaten gehören nicht in die Beispielvorlagen oder in eine Veröffentlichung.
`data/`	Erzeugte Hilfsdaten und Caches, z. B. Prognose- oder Saisonstatistiken. Inhalte können laufzeitabhängig entstehen.
`resources/icons/`	Bilddateien für PV, Haus, Netz und Batterie.
`src/core/data.db`	Standardmäßiger SQLite-Speicher für Zeitreihen; alternativ über `DASHBOARD_DB_PATH` festlegbar.

Wartung und Gesundheit

- `core/health.py`**:** hält letzten Erfolg, letzten Fehler, Fehleranzahl und Antwortzeit je Datenquelle vor.
- **Health-Tab:** zeigt den Zustand von Quellen und enthält weitere Prüf- und Verwaltungsfunktionen.
- `src/diagnose.py` **und** `tabs/diagnose_panel.py`**:** erstellen ein Diagnosepaket für die Fehlersuche.
- `core/perf_monitor.py`**:** erfasst sparsam Laufzeit-/Leistungsinformationen, insbesondere für den Raspberry-Pi-Betrieb.
- `core/offsite_backup.py`**:** unterstützt tägliche Sicherungen zu einem externen Ziel, beispielsweise über rclone.
- **Logging:** Fehler und Diagnosen werden protokolliert; beim Start richtet `main.py` zusätzlich Crash- und Debug-Logging ein.

Tests und Abhängigkeiten

Die Tests in `tests/` decken unter anderem DataStore, Zeitumrechnung, Energie-/Heizungsberechnung, Prognosen, Klima, Licht und Charts ab. Laut README können sie so gestartet werden:

```
python -m pytest tests
```

`requirements.txt` nennt unter anderem CustomTkinter, Pillow, Requests, Pandas, Matplotlib, Plotly, NumPy, psutil, Spotipy, Flask, Tado-/Hue-Bibliotheken und iCalendar-Pakete.

Start und Entwicklung

1. Python-Umgebung einrichten und Pakete aus `requirements.txt` installieren.
2. Passende Konfigurationsdateien auf Basis der Beispielvorlagen erstellen.
3. Mit `python src/main.py` oder dem passenden Startskript starten.
4. Bei Änderungen an fachlichen Berechnungen die betreffenden Tests unter `tests/` mit ausführen.

Weiterführend: `README.md` · `requirements.txt` · `tests/`

08 · MODULVERZEICHNIS

Core-Dateien auf einen Blick

Die folgende Liste ergänzt die zuvor beschriebenen Kernmodule und hilft beim Finden der zuständigen Stelle im Code.

- `core/alerts.py` – Warnungen bei PV- und Wärmeauffälligkeiten
- `core/datastore.py` – SQLite, Messwerte, Zeitreihen, Backup
- `core/schema.py` – gemeinsame Messwertschlüssel
- `core/time_utils.py` – UTC und Zeitstempel
- `core/utils.py` – allgemeine Hilfsfunktionen
- `core/health.py` – Zustand und Latenz der Datenquellen
- `core/weather.py` – Wetterclient
- `core/solar_geometry.py` – Sonnenstand und Einstrahlung
- `core/energy_day.py` – Tagesbilanz und Autarkie
- `core/ertrag_validator.py` – PV-Ertragsprüfung/-Rekonstruktion
- `core/pv_forecast.py` – PV-Modell und Erzeugungsprognose
- `core/forecast_log.py` – Prognose-Ist-Vergleich
- `core/forecast_learning.py` – Lernarchiv und Trainingsdaten
- `core/heating_events.py` – Heizereignisse erkennen
- `core/heating_stats.py` – Speicher- und Saisonstatistik
- `core/heat_demand.py` – Wärmebedarf und Solarthermie
- `core/heating_forecast.py` – Einheizempfehlung
- `core/climate.py` – gemeinsames Raumklimamodell
- `core/climate_history.py` – Raumtemperaturhistorie
- `core/climate_plan.py` – Solltemperaturplan und Status
- `core/lights.py` – testbare Lichtlogik
- `core/homeassistant.py` – Home-Assistant-API-Client
- `core/perf_monitor.py` – Leistungsprotokoll
- `core/offsite_backup.py` – externes Backup
- `core/Wechselrichter.py` – Fronius-Abfrage
- `core/BMKDATEN.py` – BMK-Abfrage und Aufbereitung

> **Wegweiser für Änderungen:** Datenabruf in `core/`, reine Berechnung ebenfalls bevorzugt in `core/`, Darstellung in `tabs/` oder `ui/`, passende Prüfung in `tests/`.

Ende der Programmdokumentation · Erstellt auf Basis von README, Quellstruktur und den zugehörigen Modulen.